

ML_HW7_111652040

Kernel Eigenfaces

1. Code with detailed explanations

Part 1.

PCA

```
def perform_pca(X, n_components):  
    mean_face = np.mean(X, axis=0)  
    X_centered = X - mean_face  
  
    # Gram matrix  
    gram_matrix = np.dot(X_centered, X_centered.T)  
    eigenvalues, eigenvectors_small = np.linalg.eigh(gram_matrix)  
  
    idx = np.argsort(eigenvalues)[::-1]  
    eigenvectors_small = eigenvectors_small[:, idx]  
  
    # Project to high dimension space  
    W_pca = np.dot(X_centered.T, eigenvectors_small[:, :n_components])  
  
    # unitlization  
    for i in range(W_pca.shape[1]):  
        W_pca[:, i] /= np.linalg.norm(W_pca[:, i])  
  
    X_pca = np.dot(X_centered, W_pca)  
    return X_pca, W_pca, mean_face
```

The goal is to reduce dimensionality by finding the directions (principal components) that maximize data variance.

1. Mean Centering: Subtract the average face (μ) from each image vector x_i . This ensures the first principal component describes the direction of maximum variance rather than the distance from the origin.
2. The "Gram Matrix" Trick: In face recognition, the number of pixels D is much larger than the number of images N . A standard covariance matrix is $D \times D$ (huge). We instead compute the Gram Matrix $K = \Phi\Phi^T$ ($N \times N$), which is computationally feasible.
3. Eigendecomposition: We solve the eigenvalue problem for the Gram Matrix. According to the Rayleigh Quotient, the eigenvectors associated with the largest eigenvalues represent the directions of maximum variance.

4. Back-Projection & Orthonormalization: We project the small eigenvectors back into the high-dimensional space to get the "Eigenfaces" (W_{pca}) and normalize them to unit length (L_2 norm) so they form an orthonormal basis.

LDA

```
def perform_lda(X, y, n_components):
    n_samples, n_features = X.shape
    class_labels = np.unique(y)
    mean_overall = np.mean(X, axis=0)
    X_centered = X - mean_overall

    Kw = np.zeros((n_samples, n_samples))
    Kb = np.zeros((n_samples, n_samples))

    for c in class_labels:
        idx_c = (y == c)
        X_c = X[idx_c]
        n_c = X_c.shape[0]
        mean_c = np.mean(X_c, axis=0)

        v_b = np.dot(X_centered, (mean_c - mean_overall).T).reshape(-1, 1)
        Kb += n_c * np.dot(v_b, v_b.T)

        v_w = np.dot(X_centered, (X_c - mean_c).T)
        Kw += np.dot(v_w, v_w.T)

    Kw += np.eye(n_samples) * 1e-6
    A = np.dot(np.linalg.pinv(Kw), Kb)
    eigenvalues, alphas = np.linalg.eig(A)

    idx = np.argsort(eigenvalues.real)[::-1]
    alphas = alphas.real[:, idx[:, :n_components]]
    W_lda = np.dot(X_centered.T, alphas)

    for i in range(W_lda.shape[1]):
        W_lda[:, i] /= np.linalg.norm(W_lda[:, i])

    X_lda = np.dot(X_centered, W_lda)
    return X_lda, W_lda
```

The goal is to reduce dimensionality by finding the projection that minimize between-class scatter while minimize within-class scatter

1. Global and Class-wise Centering: The code calculates a mean_overall to center the entire dataset and a mean_c for each specific class to understand how individual groups deviate from the global average.
2. Constructing Scatter Matrices (S_B and S_W):
 - * Between-class scatter (K_b): It captures how far the class means are from the global mean, representing the signal we want to maximize.
 - Within-class scatter (K_w): It captures the spread of data within each class, representing the "noise" we want to minimize.
3. Regularization: A small value (1e-6) is added to the diagonal of K_w to ensure the matrix is non-singular and invertible before solving the eigenvalue problem.

4. Solving the Generalized Eigenvalue Problem: The code computes $A = S_w^{-1}S_b$ and finds its eigenvalues to determine the most discriminative directions.
5. Projection and Normalization: The resulting vectors are mapped back to the original feature space and normalized to ensure they are unit vectors (L_2 norm), forming the final "Fisherfaces".

Part 2.

```
def testing(train_X, train_y, test_X, test_y, W, mean_face, method_name, k=3, img_shape=(231, 195)):
    recog_dir = f'experiments/{method_name}/recognition/'
    recon_dir = f'experiments/{method_name}/reconstruction/'
    for d in [recog_dir, recon_dir]:
        if not os.path.exists(d): os.makedirs(d)

    train_features = np.dot(train_X - mean_face, W)
    test_features = np.dot(test_X - mean_face, W)

    predictions, closest_idx = knn_predict(train_features, train_y, test_features, k=k)
    accuracy = np.mean(predictions == test_y)
    print(f"[{method_name.upper()}] Accuracy: {accuracy * 100:.2f}%")

    test_X_reconstructed = np.dot(test_features, W.T) + mean_face

    sample_idx = np.random.choice(len(test_X), min(10, len(test_X)), replace=False)
    for i in sample_idx:
        plt.figure(figsize=(8, 4))
        plt.subplot(1, 2, 1); plt.imshow(test_X[i].reshape(img_shape), cmap='gray'); plt.axis('off')
        plt.subplot(1, 2, 2); plt.imshow(train_X[closest_idx[i]].reshape(img_shape), cmap='gray'); plt.axis('off')
        plt.savefig(os.path.join(recog_dir, f"recog_{i}.png")); plt.close()

        plt.figure(figsize=(8, 4))
        plt.subplot(1, 2, 1); plt.imshow(test_X[i].reshape(img_shape), cmap='gray'); plt.axis('off')
        plt.subplot(1, 2, 2); plt.imshow(test_X_reconstructed[i].reshape(img_shape), cmap='gray'); plt.axis('off')
        plt.savefig(os.path.join(recon_dir, f"recon_{i}.png")); plt.close()
```

1. Face Recognition (Classification)

The recognition part of the function focuses on identifying the person in a test image by comparing it to the training database.

- Feature Projection: The test images are first projected into the reduced feature space using the weight matrix W .
- Similarity Matching: The `knn_predict` function compares the features of a test image against all `train_features` to find the k closest neighbors.
- Accuracy Metrics: The code calculates the mean accuracy by checking how many predictions match the ground truth `test_y`.
- Visual Output: The function saves a comparison image (`recog_{i}.png`) showing the original test face side-by-side with the closest match found in the training set.

2. Face Reconstruction (Visualization)

The reconstruction part evaluates how much facial information is preserved in the reduced subspace by trying to rebuild the original image.

- Inverse Mapping: The code performs an inverse projection by multiplying the test_features by the transpose of the weight matrix (W^T) and adding back the mean_face.
- Mathematical Formula: The reconstruction is calculated as $X_{reconstructed} = (X_{projected} \cdot W^T) + \mu$.
- Visual Comparison: The function saves a comparison image (recon_{i}.png) showing the original test image and its reconstructed version.
- Quality Assessment: This allows you to see if the chosen $n_components$ are sufficient to capture the unique features (like eyes, nose, or mouth) of the subject's face.

Part 3.

kernel PCA

```
def perform_kpca(X, n_components, kernel_type='rbf', gamma=None):
    N = X.shape[0]
    K = compute_kernel(X, X, kernel_type, gamma)
    one_n = np.ones((N, N)) / N
    K_centered = K - one_n.dot(K) - K.dot(one_n) + one_n.dot(K).dot(one_n)

    eigenvalues, eigenvectors = np.linalg.eigh(K_centered)
    idx = np.argsort(eigenvalues)[::-1]
    alphas = eigenvectors[:, idx[:n_components]]
    lambdas = eigenvalues[idx[:n_components]]

    for i in range(alphas.shape[1]):
        if lambdas[i] > 1e-10:
            alphas[:, i] /= np.sqrt(lambdas[i])

    return alphas, lambdas
```

1. Kernel Computation: The code uses a kernel function (like RBF) to compute the similarity matrix K , representing the dot products of data points in a higher-dimensional feature space.
2. Kernel Centering: Because we cannot center the data in the implicit feature space directly, the code applies a "double-centering" formula to K using an $N \times N$ matrix of ones (one_n).
3. Eigendecomposition of the Kernel: The code solves for the eigenvalues and eigenvectors of the centered kernel matrix. These eigenvectors (α)

represent the coefficients of the principal components.

4. Eigenvector Scaling: The eigenvectors are scaled by the square root of their corresponding eigenvalues ($1/\sqrt{\lambda}$) to ensure they are properly normalized in the feature space.

kernel LDA

```
def perform_klda(X, y, n_components, kernel_type='rbf', gamma=None):
    N = X.shape[0]
    labels = np.unique(y)
    K = compute_kernel(X, X, kernel_type, gamma)

    Z = np.zeros((N, N))
    for label in labels:
        indices = np.where(y == label)[0]
        Z[np.ix_(indices, indices)] = 1.0 / len(indices)

    M = K.dot(Z).dot(K)
    N_mat = K.dot(K) + np.eye(N) * 1e-3

    eigenvalues, alphas = np.linalg.eig(np.linalg.pinv(N_mat).dot(M))
    idx = np.argsort(eigenvalues.real)[::-1]
    alphas = alphas.real[:, idx[:n_components]]

    return alphas
```

1. Kernel Projection: Similar to KPCA, it first computes the Kernel matrix K to map the data into a high-dimensional space implicitly.
2. Class-weighting Matrix (Z): The code builds a matrix Z that acts as a class-wise mean-subtraction tool within the kernel space, helping to define the between-class and within-class variations.
3. Matrix Construction (M and N):
 - * M : Represents the between-class scatter in the kernel space.
 - N_{mat} : Represents the total scatter in the kernel space, with a small regularization term ($1e-3$) added for numerical stability.
4. Eigenvalue Optimization: By solving the problem for $N_{mat}^{-1}M$, the code finds the directions that best separate the classes in the non-linear feature space.

2. Experiments and Discussion

Part 1: reconstruction

hyperparameter: n_components = 30

PCA	LDA
 	 

Part 2: recognition

PCA	LDA
Accuracy: 83.33%	Accuracy: 86.67%
 	 

Observations and Discussion: Topic 1: PCA vs. LDA and the Role of Class Labels

The fundamental difference in accuracy between PCA (83.33%) and LDA (86.67%) stems from how they handle class-specific information.

- **PCA is Unsupervised:** PCA focuses purely on finding the directions of maximum variance in the data without considering labels. In the Yale Face Database, large variances often come from lighting changes (e.g., rightlight) or expressions (e.g., wink), which PCA treats as "important" features even if they don't help identify the person.
- **LDA is Supervised:** LDA uses class labels to distinguish between different people. It specifically works to maximize between-class variance (pushing different people apart) while minimizing within-class variance.

- Handling Variations: Because LDA learns from labels, it correctly understands that a person with glasses and that same person without glasses belong to the same group. It "squashes" these variations together, whereas PCA might separate them into different groups because they look mathematically different at the pixel level.

Part 3

[KPCA] Kernel: rbf, Accuracy: 83.33%

[KLDA] Kernel: rbf, Accuracy: 90.00%

[KPCA] Kernel: poly, Accuracy: 83.33%

[KLDA] Kernel: poly, Accuracy: 93.33%

[KPCA] Kernel: sigmoid, Accuracy: 83.33%

[KLDA] Kernel: sigmoid, Accuracy: 93.33%

Observations and Discussion: Topic 2: Higher Dimensional Mapping via the Kernel Trick

The results show that moving from linear models to Kernel LDA (KLDA) further boosted accuracy to 93.33%.

- The Problem of Non-Linearity: Real-world data, especially facial images with different poses and occlusions, is often not linearly separable in its original space.
- Higher Dimensional Projection: The "Kernel Trick" (using kernels like RBF or Poly) implicitly maps the data into a much higher-dimensional feature space. According to Cover's Theorem, complex patterns are more likely to become linearly separable when projected into a higher dimension.
- Improved Classification: By combining this higher-dimensional projection with LDA's supervised learning, KLDA is able to find complex, non-linear boundaries that can't be captured by standard linear methods. This

allowed your model to reach its peak accuracy of 93.33%, successfully untangling the difficult cases that both PCA and standard LDA missed.

B. t-SNE

1. Code with detailed explanations

Part 1.

```
def run_embedding(X, labels, method='tsne', no_dims=2, perplexity=30.0):
    X = pca(X, 50).real
    (n, d) = X.shape
    max_iter = 500
    Y = np.random.randn(n, no_dims)
    iY = np.zeros((n, no_dims))
    gains = np.ones((n, no_dims))
    frames = []

    P = x2p(X, 1e-5, perplexity)
    P = (P + P.T) / (2 * n)
    P = np.maximum(P * 4, 1e-12) # Early exaggeration

    for iter in range(max_iter):
        sum_Y = np.sum(np.square(Y), 1)
        dist = np.add(np.add(-2 * np.dot(Y, Y.T), sum_Y).T, sum_Y)

        if method == 'tsne':
            num = 1. / (1. + dist) # t-分布
        else:
            num = np.exp(-dist) # 高斯分布 (Symmetric SNE)

        np.fill_diagonal(num, 0)
        Q = num / np.sum(num)
        Q = np.maximum(Q, 1e-12)

        # Gradient
        PQ = P - Q
        dY = np.zeros((n, no_dims))
        if method == 'tsne':
            for i in range(n):
                dY[i, :] = np.sum(np.tile(PQ[:, i] * num[:, i], (no_dims, 1)).T * (Y[i, :] - Y), 0)
        else:
            for i in range(n):
                dY[i, :] = np.sum(np.tile(PQ[:, i], (no_dims, 1)).T * (Y[i, :] - Y), 0)

        # Update
        momentum = 0.5 if iter < 20 else 0.8
        gains = (gains + 0.2) * ((dY > 0.) != (iY > 0.)) + (gains * 0.8) * ((dY > 0.) == (iY > 0.))
        gains[gains < 0.01] = 0.01
        iY = momentum * iY + 500 * (gains * dY)
        Y = Y + iY
        Y = Y - np.mean(Y, 0)

        if iter == 100: P = P / 4.

        if iter % 10 == 0:
            fig, ax = plt.subplots(figsize=(6, 6))
            ax.scatter(Y[:, 0], Y[:, 1], 10, labels, cmap='tab10')
            ax.set_title(f"{method.upper()} Iteration {iter}")
            fig.canvas.draw()
            image = np.frombuffer(fig.canvas.tostring_rgb(), dtype='uint8')
            frames.append(image.reshape(fig.canvas.get_width_height()[::-1] + (3,)))
            plt.close()

    save_gif(frames, f"{method}_optimization.gif")
    return Y, P, Q
```

The run_embedding() function implements Symmetric-SNE and t-SNE

Step-by-Step Logic of run_embedding()

1. Initial Dimensionality Reduction: The function first reduces the data to 50 dimensions using standard PCA to remove noise and speed up the computation of pairwise distances.

2. High-Dimensional Similarity (P): It calculates a probability distribution P that represents the similarity between points in the original high-dimensional space.
3. Early Exaggeration: The code multiplies P by 4 during early iterations ($P = \text{np.maximum}(P * 4, 1e-12)$). This forces clusters of points to form tight groups, making it easier for them to move around each other to find an optimal global organization.
4. Low-Dimensional Similarity (Q): In each iteration, it calculates similarities Q between points in the low-dimensional embedding (Y). The choice of distribution depends on the method:
 - t-SNE: Uses a Student t-distribution (Cauchy distribution), which has "heavy tails" to solve the crowding problem.

$$q_{ij} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq l} \exp(-\|y_k - y_l\|^2)}$$

- Symmetric SNE: Uses a Gaussian (Normal) distribution.

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$

5. Gradient Descent Optimization: The code calculates the gradient (difference between P and Q) and updates the positions of points in Y using momentum and adaptive gains to reach a stable visualization.

Part 2

```
def save_gif(frames, filename):
    print(f"Saving GIF to {filename}...")
    imageio.mimsave(filename, frames, fps=10)

def plot_similarity(P, Q, title):
    plt.figure(figsize=(12, 4))
    p_flat = P[P > 1e-12].flatten()
    q_flat = Q[Q > 1e-12].flatten()

    plt.subplot(1, 2, 1)
    plt.hist(np.log10(p_flat), bins=50, color='blue', alpha=0.6)
    plt.title(f"{title}: High-dim P (Log10)")

    plt.subplot(1, 2, 2)
    plt.hist(np.log10(q_flat), bins=50, color='red', alpha=0.6)
    plt.title(f"{title}: Low-dim Q (Log10)")
    plt.savefig(f"similarity_{title.lower()}.png")
    plt.show()
```

The block within `run_embedding()` that triggers every 10 iterations is designed to capture the dynamic evolution of the data points as they find their optimal positions in the 2-dimensional subspace.

1. **Creation:** For every 10th iteration, the code initializes a new matplotlib figure of size 6x6 inches to prevent overlapping previous frames. Scatter
2. **Plotting:** The current coordinates in the low-dimensional embedding (Y) are plotted. The labels array is used to color-code the points by their original class, allowing you to visually assess if the algorithm is successfully clustering similar faces together.
3. **Title and State:** The plot title dynamically updates with the current iteration number, which is crucial for tracking the speed of convergence. **Frame Buffering:** Instead of just showing a plot, `fig.canvas.tostring_rgb()` converts the plot into a raw pixel buffer. This buffer is reshaped into an image array and appended to the frames list. **GIF**
4. **Generation:** After the loop finishes, all captured frames are saved as a GIF using `save_gif()`. This provides a time-lapse of how "early exaggeration" pushes clusters apart before they settle into a stable configuration.

Part 3 .

```
def save_gif(frames, filename):
    print(f"Saving GIF to {filename}...")
    imageio.mimsave(filename, frames, fps=10)

def plot_similarity(P, Q, title):
    plt.figure(figsize=(12, 4))
    p_flat = P[P > 1e-12].flatten()
    q_flat = Q[Q > 1e-12].flatten()

    plt.subplot(1, 2, 1)
    plt.hist(np.log10(p_flat), bins=50, color='blue', alpha=0.6)
    plt.title(f"{title}: High-dim P (Log10)")

    plt.subplot(1, 2, 2)
    plt.hist(np.log10(q_flat), bins=50, color='red', alpha=0.6)
    plt.title(f"{title}: Low-dim Q (Log10)")
    plt.savefig(f"similarity_{title.lower()}.png")
    plt.show()
```

The `plot_similarity(P, Q, title)` function is a diagnostic tool used to evaluate how well the low-dimensional distribution (Q) matches the high-dimensional distribution (P).

1. **Data Flattening and Filtering:** The code flattens the $N \times N$ similarity matrices and filters out values near zero ($< 10^{-12}$) to focus on meaningful relationships.

2. Logarithmic Scale ($\log_{10}$): Because similarity values often span several orders of magnitude, the code plots the base-10 logarithm. This makes it easier to visualize the "heavy tails" of the distribution.
3. High-dim P (Blue): This histogram shows the distribution of similarities in the original space. A spike on the right indicates many points are very close together (neighbors), while a long tail to the left represents distant points.
4. Low-dim Q (Red): This histogram shows the distribution in the 2D/3D map.

Part 4 .

```
def Hbeta(D=np.array([]), beta=1.0):
    P = np.exp(-D.copy() * beta)
    sumP = np.sum(P)
    H = np.log(sumP) + beta * np.sum(D * P) / sumP
    P = P / sumP
    return H, P
```

```
def x2p(X=np.array([]), tol=1e-5, perplexity=30.0):
    print("Computing pairwise distances...")
    (n, d) = X.shape
    sum_X = np.sum(np.square(X), 1)
    D = np.add(np.add(-2 * np.dot(X, X.T), sum_X).T, sum_X)
    P = np.zeros((n, n))
    beta = np.ones((n, 1))
    logU = np.log(perplexity)

    for i in range(n):
        if i % 500 == 0:
            print("Computing P-values for point %d of %d..." % (i, n))
            betamin = -np.inf
            betamax = np.inf
            Di = D[i, np.concatenate((np.r_[0:i], np.r_[i+1:n]))]
            (H, thisP) = Hbeta(Di, beta[i])

            Hdiff = H - logU
            tries = 0
            while np.abs(Hdiff) > tol and tries < 50:
                if Hdiff > 0:
                    betamin = beta[i].copy()
                    if betamax == np.inf or betamax == -np.inf:
                        beta[i] = beta[i] * 2.
                    else:
                        beta[i] = (beta[i] + betamax) / 2.
                else:
                    betamax = beta[i].copy()
                    if betamin == np.inf or betamin == -np.inf:
                        beta[i] = beta[i] / 2.
                    else:
                        beta[i] = (beta[i] + betamin) / 2.
                (H, thisP) = Hbeta(Di, beta[i])
                Hdiff = H - logU
                tries += 1
            P[i, np.concatenate((np.r_[0:i], np.r_[i+1:n]))] = thisP
    return P
```

In the context of t-SNE, the functions `Hbeta()` and `x2p()` work together to define the "neighborhood" of each data point by calculating probabilities in the high-dimensional space.

1. Hbeta(D, beta) Explanation

- This function calculates the Gaussian kernel probabilities and the Shannon entropy for a single data point relative to all others based on

a specific precision value (β).

- Gaussian Probabilities: It computes $P = \exp(-D \cdot \beta)$, where D is a vector of squared Euclidean distances from the current point to all other points.
- Normalization: The raw values in P are divided by their sum (sumP) to ensure the resulting probabilities for that point sum to 1.
- Entropy Calculation: It calculates the entropy H of the distribution, which measures how "spread out" the probability is. If β is large, the distribution is concentrated on the nearest neighbors (low entropy); if β is small, the distribution is spread across many points (high entropy).

2. x2p(X, tol, perplexity) Explanation

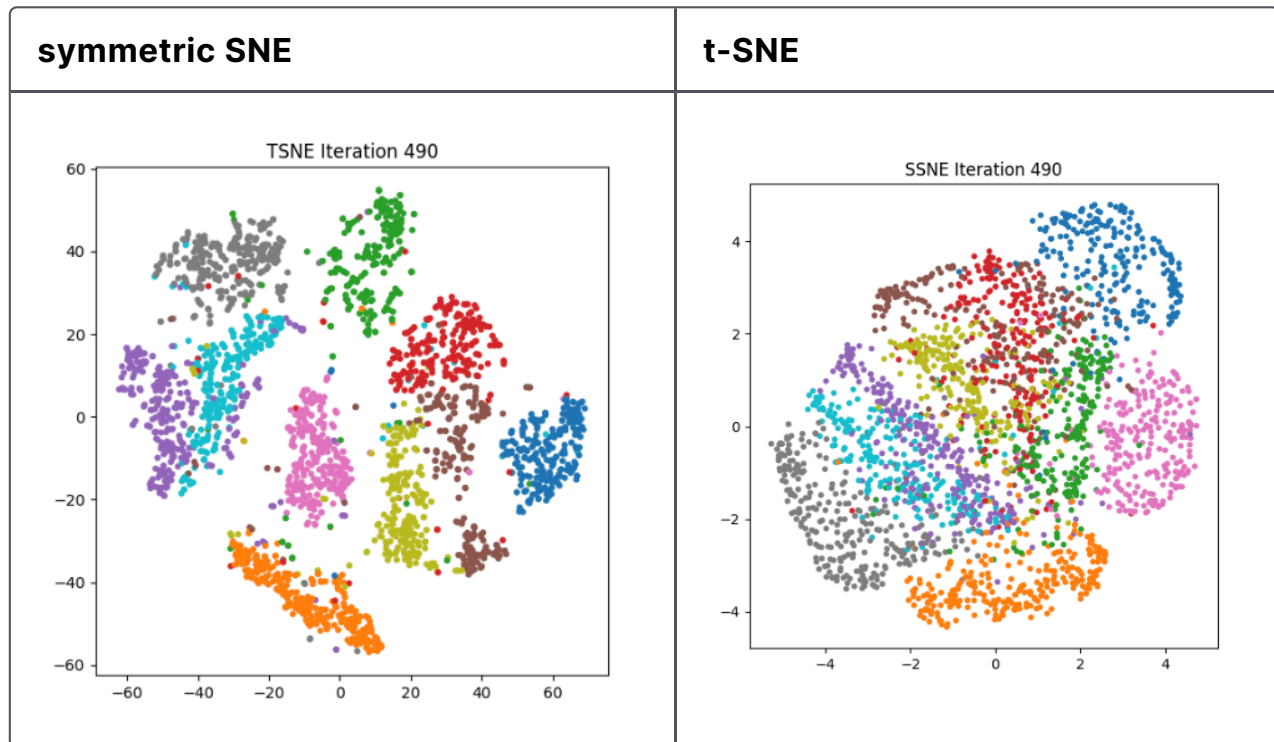
This is the main driver function that determines the high-dimensional probability matrix P for the entire dataset.

- Pairwise Distances: It first calculates the squared Euclidean distance between every pair of points in the dataset X .
- Target Perplexity: It converts the user-defined perplexity into a target log-entropy value ($\log U$). Perplexity can be thought of as a smooth measure of the effective number of neighbors.
- Binary Search for β : For every individual point i , the function performs a binary search to find the optimal β_i :
 - It tests a β value using `Hbeta()` and compares the resulting entropy to the target $\log U$.
 - If the entropy is too high, it increases β (shrinks the neighborhood); if too low, it decreases β (expands the neighborhood).
 - The loop continues until the difference is within a specified tolerance (tol) or it reaches 50 attempts.
- Symmetrization: After the loop, the code (in the calling function `run_embedding`) typically ensures P is symmetric so that the

similarity of point A to B is the same as B to A.

2. Experiments and Discussion

Part 1, Part 2



Observations and Discussion: Topic 1 : The crowding problem

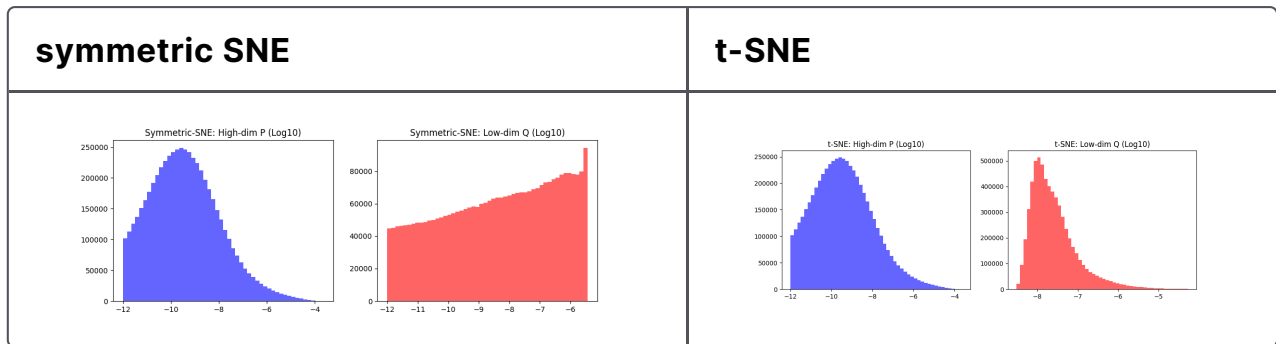
The Crowding Problem is a fundamental challenge when visualizing high-dimensional data in 2D or 3D spaces.

- **Dimensionality Mismatch:** In high-dimensional space, there is more room for points to be far apart while still being neighbors; however, in 2D space, there is not enough area to preserve all these distances accurately.
- **The Student t-Distribution Solution:** t-SNE addresses this by using a heavy-tailed Student t-distribution for the low-dimensional space (the red Q plots in your experiment).
- **Pushing Clusters Apart:** This heavy tail allows points that are moderately far apart in the high-dimensional space to be pushed even further apart in the 2D map. This prevents all data points from collapsing into a single

dense "blob" in the center of the plot, effectively creating the distinct clusters seen in successful visualizations.

-

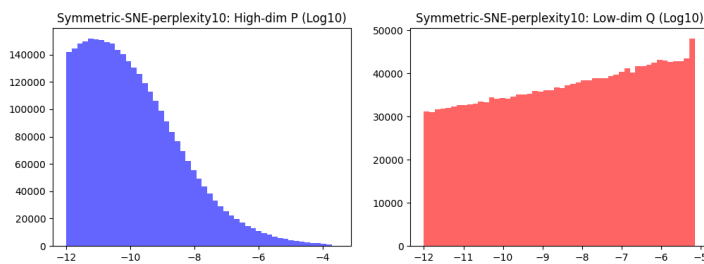
Part 3



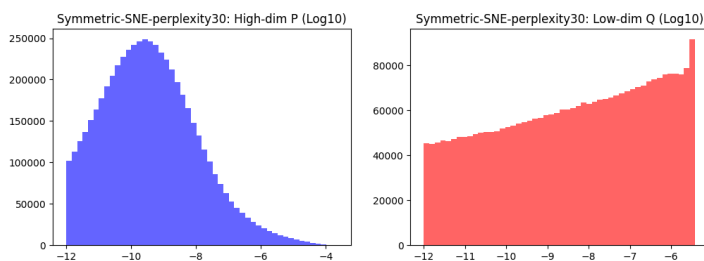
Part 4

symmetric SNE

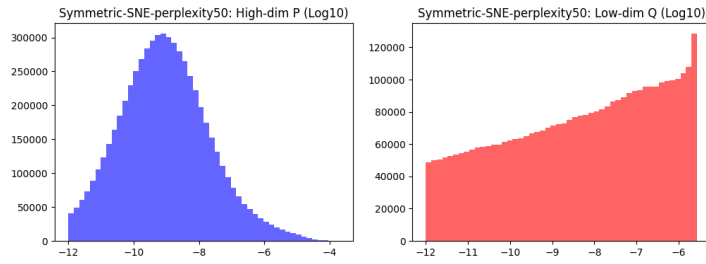
- perplexity 10



- perplexity 30

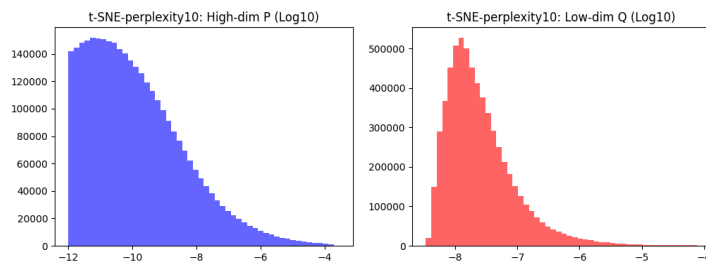


- perplexity 50

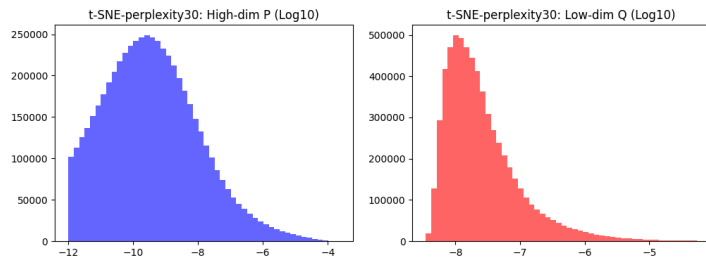


t-SNE

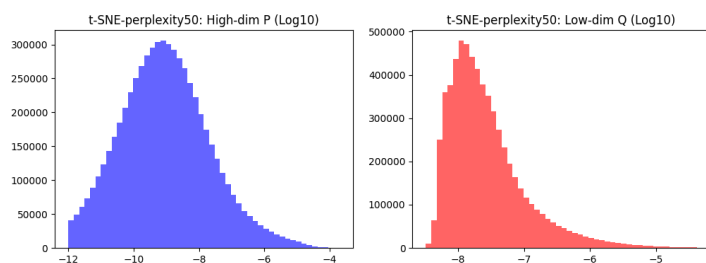
- perplexity 10



- perplexity 30



- perplexity 50



Observations and Discussion: Topic 2: Impact of Perplexity on High-Dimensional P

The blue histograms represent the High-dimensional joint probabilities (P). As you observed, the shift in these distributions as perplexity increases from 10 to 50 is quite distinct.

- Entropy and Neighborhood Size: Perplexity is essentially a target for the Shannon entropy of the distribution. A higher perplexity (e.g., 50) forces the model to consider more points as "neighbors".
 - The Rightward Shift (Higher Log-Probabilities): As perplexity increases, the peak of the blue distribution moves from left to right (e.g., from -11 toward -9). This happens because the algorithm increases the bandwidth (σ_i) of the Gaussian kernels to encompass more neighbors.
- Global vs. Local Focus:
- * Low Perplexity (10): The distribution is "colder" with lower probabilities, meaning the model only cares about very immediate neighbors, often resulting in fragmented clusters.
 - High Perplexity (50): The shift toward higher probabilities indicates a "warmer" distribution that captures the broader, global structure of the dataset, allowing the model to see the "big picture" of how subjects are related.